WEB APPLICATION BASICS

TryHackMe Learning & Practical Report

Prepared by: Faijan Ansari

Focus: Web Application Security / WAPT Foundations

Platform: TryHackMe — Cyber Security 101

Room: Web Application Basics

A structured technical report documenting the concepts studied and practical HTTP exercises completed in the Web Application Basics room.

Ai Generated & Only for understand the F ucture

Prepared for learning documentation, portfolio use, and cybersecurity training.

1. Executive Summary

I completed the TryHackMe **Web Application Basics** room as part of my web application security learning path. The room focuses on the communication model behind modern web applications: how browsers and web servers interact, how URLs are structured, how HTTP requests and responses are formed, and how security-related headers and cookie attributes contribute to application security.

The practical portion reinforced these concepts by working with API endpoints using different HTTP methods. The exercises demonstrated how GET can retrieve information, POST can modify application data, and DELETE can remove a resource. This connected protocol-level knowledge with request manipulation commonly encountered during web application testing.

The room also introduced defensive concepts including Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), MIME-type protection, Secure cookies, and HttpOnly cookies.

2. Lab Information

Platform	TryHackMe
Room	Web Application Basics
Learning Path	Cyber Security 101
Primary Domain	Web Application Security
Purpose	Foundational
Completion	Completed
Primary Skills	HTTP, URLs, requests, responses, headers, cookies, API methods

Room: https://tryhackme.com/room/webapplicationbasics

Supporting reference: TRedEye, “Web Application Basics — Tryhackme Walkthrough” (Medium, Nov. 1, 2024). This report is independently structured and paraphrased for learning documentation.

AI Generated & Only for understand the Folder Structure

3. Learning Objectives

The main objectives were to understand the fundamentals that sit underneath web application penetration testing:

- Identify the main components involved in delivering a web application.
- Understand URL structure and the purpose of query strings.
- Understand HTTP request and response messages.
- Recognize common HTTP request methods and their intended use.
- Understand request/response headers, bodies, and status codes.
- Identify security-related cookie attributes and HTTP security headers.
- Perform basic API interaction using GET, POST, and DELETE requests.

4. Web Application Architecture Basics

A web application generally involves a client, a server, and network communication between them. The **web browser** acts as the client that sends requests and renders returned content. The **web server** receives requests and delivers resources or application responses. A **Web Application Firewall (WAF)** can operate as a protective layer that examines incoming traffic and may block requests associated with known malicious patterns.

From a penetration-testing perspective, understanding these components is important because each layer can expose a different attack surface. A tester may examine how the browser constructs requests, how the server handles unexpected input, and whether defensive controls correctly restrict suspicious traffic.

5. URL Structure and Security Relevance

A Uniform Resource Locator (URL) identifies a resource and provides the information needed to access it. Important components include the scheme/protocol, hostname, path, and optional query string. HTTPS provides encrypted communication between the client and server.

Query strings commonly carry parameters such as search terms, identifiers, filters, or application input. During security testing, understanding how changing a parameter affects application behavior is essential because parameters are ultimately controlled by the client.

Typosquatting refers to registering domains that resemble legitimate domains through common spelling variations. The security relevance is that users may accidentally visit an attacker-controlled domain.

6. HTTP Messages

HTTP communication is based on messages exchanged between the client and server. A client sends an **HTTP request**; after processing it, the server returns an **HTTP response**. These messages contain structured sections such as a request/status line, headers, an empty line separating headers from the body, and an optional message body.

This structure is fundamental to web application testing. Tools such as Burp Suite make it possible to intercept, inspect, modify, and resend these messages, allowing a tester to observe how an application responds to controlled changes.

7. HTTP Requests: Methods and Components

HTTP methods communicate the intended action for a request. The room introduced methods including GET, POST, and OPTIONS. GET is commonly used to retrieve resources, POST is commonly used to submit data or trigger a state-changing operation, while OPTIONS can be used to discover communication options supported by a target resource.

The request line identifies the method, target path, and HTTP version. The **Host** header identifies the destination host. Other headers can describe the client, accepted formats, authentication information, cookies, or the type of data being sent.

For form-style data encoded as key/value pairs, **application/x-www-form-urlencoded** is a common content type. Understanding content types is important because the server may parse request bodies differently depending on the declared format.

8. HTTP Responses and Status Codes

An HTTP response contains a status line followed by response headers and, when applicable, a response body. The status line communicates the HTTP version, status code, and a short explanation.

Status codes provide a quick indication of how the server handled a request. For example, **404 Not Found** indicates that the requested resource could not be found. Server-error responses belong to the 5xx class and indicate that the server encountered a condition preventing it from successfully completing the request.

During penetration testing, status codes are useful signals. Differences between 2xx, 3xx, 4xx, and 5xx responses can help identify whether an endpoint exists, whether authentication is required, whether input was accepted, or whether an application encountered an error.

AI Generated & Only for understand

9. Response Headers, Cookies, and Security

Response headers can disclose information or enforce browser-side security behavior. The **Server** header may expose information about server software, contributing to technology fingerprinting. Reducing unnecessary version disclosure is one part of defensive hardening.

Cookies can include security attributes that reduce exposure. The **Secure** attribute instructs the browser to send a cookie only over HTTPS. The **HttpOnly** attribute prevents client-side JavaScript from directly reading the cookie, which can reduce the impact of certain XSS scenarios involving cookie theft.

These controls are defense-in-depth mechanisms and do not replace secure authentication, authorization, session management, and input handling.

10. Security Headers

Content-Security-Policy (CSP) can restrict where browser resources such as scripts are allowed to load from. A directive such as **script-src** controls permitted script sources.

HTTP Strict Transport Security (HSTS) instructs browsers to use HTTPS for a site for a specified period. The **includeSubDomains** directive extends the policy to subdomains.

X-Content-Type-Options: nosniff helps prevent browsers from MIME-sniffing a response and interpreting content as a different type than the server declared.

Ai Generated & Only for understand the Folder Structure

11. Practical Task — API Requests

The practical exercise connected the protocol theory to real HTTP interactions. The lab provided an API and required different HTTP methods to interact with user resources.

11.1 GET Request — Retrieving Data

The first action was a GET request to **/api/users** to retrieve the available user list. This demonstrated the typical use of GET for reading server-side resources.

```
GET /api/users
```

The lab confirmed successful retrieval with the corresponding challenge flag. The broader security lesson is that an endpoint should expose only information the requesting user is authorized to access. In real applications, API enumeration and excessive data exposure are important testing areas.

11.2 POST Request — Modifying Data

The second exercise required a POST request to **/api/user/2** to update Bob's country from UK to US. This demonstrated that HTTP requests can cause state changes and that the request body can carry application data.

```
POST /api/user/2
country=US
```

The key lesson is that state-changing endpoints require strong server-side authorization and input validation. A secure application should verify that the authenticated user is permitted to modify the selected resource instead of trusting a resource identifier supplied by the client.

11.3 DELETE Request — Removing Data

The final exercise used DELETE against **/api/user/1** to remove a user. This demonstrated another state-changing HTTP method and reinforced the importance of authorization checks for destructive operations.

```
DELETE /api/user/1
```

In production, deletion functionality should be protected by authentication, authorization, validation, logging, and—where appropriate—confirmation or recovery controls.

Ai Generated Only for use in the Folder Structure

12. Key Security Takeaways

The most valuable outcome was understanding that web application security begins with understanding normal web communication. Before testing complex vulnerabilities, a penetration tester needs to be comfortable reading and manipulating HTTP traffic.

- HTTP methods communicate intended operations, but the server must enforce authorization independently.
- URL parameters and request bodies are user-controlled input and should not be blindly trusted.
- Status codes and response differences provide useful signals during testing.
- Security headers provide browser-side protections and should be reviewed during assessments.
- Secure and HttpOnly cookie attributes reduce certain risks but do not replace proper session security.
- API endpoints should enforce authentication, authorization, input validation, and appropriate data exposure.
- Understanding requests and responses is essential preparation for tools such as Burp Suite.

13. Skills Practiced

Area	Practical understanding gained
Web fundamentals	Client/server architecture and application delivery

Area	Practical understanding gained
URLs	Protocol, host, path, query strings, and security relevance
HTTP	Request/response structure and HTTP/1.1 concepts
Methods	GET, POST, OPTIONS, and DELETE use cases
Headers	Request and response metadata
Status codes	Interpreting success, client-error, and server-error responses
Cookies	Secure and HttpOnly security attributes
Security headers	CSP, HSTS, and nosniff
API testing	Basic resource retrieval, modification, and deletion

14. What I Would Explore Next

This room establishes the foundation for deeper Web Application Penetration Testing. My next steps are to apply the HTTP knowledge to authentication and session management, access control and IDOR, input validation, SQL injection, cross-site scripting, file inclusion, command injection, SSRF, API security, and business-logic vulnerabilities.

A particularly useful next step is becoming comfortable with Burp Suite: intercepting requests, changing parameters, replaying requests, comparing responses, comparing responses, and documenting findings. These skills build directly on the request/response model covered here.

15. Conclusion

Completing Web Application Basics gave me a clearer understanding of how web applications communicate at the HTTP level. Instead of viewing a website only as a visual interface, I learned to view it as a collection of requests, responses, resources, parameters, headers, cookies, and server-side actions.

The practical API tasks were especially useful because they demonstrated how HTTP methods can retrieve, modify, and delete application data. This foundation will help me approach future WAPT labs systematically: first understand normal behavior, then identify where trust boundaries, authorization, input handling, or security controls may fail.

Completion status: Successfully completed — TryHackMe Web Application Basics.

nerated & Only for understand the Folder structure

16. References

1. TryHackMe — Web Application Basics: https://tryhackme.com/room/webapplicationbasics

2. TRedEye — Web Application Basics — Tryhackme Walkthrough, Medium, Nov. 1, 2024: https://medium.com/@TRedEye/web-application-basics-tryhackme-walkthrough-ee6a0cf2f1f6

Note: The Medium article was used as a supporting reference for room structure and terminology. This report is an original learning document and does not reproduce the article verbatim.